

pandas 데이터 파악과 조작

분석할 데이터를 수집(확보)하면 데이터의 특징을 파악하고 다루기 쉽게 변형하는 작업을 수행해야 한다

#2. 데이터 조작(가공)

- 데이터 개수 세기 : `count()`, `value_counts()`
- 데이터 정렬 : `sort_values()`, `sort_index()`
- 데이터 순위 : `rank()`
- 데이터 집계 : 합계(`sum()`), 평균(`mean()`), 최대(`max()`), 최소(`min()`)
- 데이터 삭제 : `drop(axis=0/1)`
- 결측치 처리 : `dropna(axis=0/1, subset, inplace)`
- 중복값 처리 : `duplicated()`, `drop_duplicates()`
- 데이터 변경 :
 - 단위환산
 - 자료형 변경 : `astype()`
 - 수치형 데이터를 범주형 데이터로 변경 :
 - 구간을 지정하여 범주화 : `cut(data, bins, labels)`
 - 동일한 개수를 갖도록 범주화 : `qcut(data, bins_num, labels)`
 - 스케일링(`scaling`) : 정규화(`min-max scaling`), 표준화(`z-score`)
 - 시계열데이터 처리
- 행/열에 동일한 함수 적용 : `apply()`, `applymap()`, `map()`

```
In [1]: import numpy as np
import pandas as pd
print(f'Numpy:{np.__version__}, Pandas:{pd.__version__}')
```

Numpy:2.5.2, Pandas:3.0.5

```
In [2]: from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
```

7. 행/열에 동일한 연산 적용 `apply()`

`apply()` 함수

- DataFrame의 행이나 열에 복잡한 연산을 `vectorizing` 할 수 있게 해주는 함수
- 매우 많이 활용되는 함수

형식 : `apply(반복적용할 함수, axis=0/1)`

- 0 : 열마다 반복
- 1 : 행마다 반복
- 생략시 기본값 : 0

```
Series.apply(func, args=(), *, by_row='compat', **kwargs)
DataFrame.apply(func, axis=0, raw=False, result_type=None, args=(),
```

사용 목적

- 사용자 정의함수(User Defined Function:UDF)를 시리즈나 데이터프레임에 적용
- 복합 조건 처리 : 여러 컬럼을 동시에 참조하는 계산
- 집계 및 변환 : 그룹별계산, 열반환
- 비벡터화 연산 처리 : pandas 기본연산으로 처리하기 어려운 행/열 단위 로직 처리

예제

```
In [3]: df = pd.DataFrame({'a':[1,3,4,3,4],
                           'b':[2,3,1,4,5],
                           'c':[1,5,2,4,4]})
df
```

```
Out[3]:
```

	a	b	c
0	1	2	1
1	3	3	5
2	4	1	2
3	3	4	4
4	4	5	4

데이터프레임의 각 열에 sum() 함수 적용

apply(함수, axis=0)

```
In [4]: df
df.apply(np.sum, axis=0)
```

```
Out[4]:
```

	a	b	c
0	1	2	1
1	3	3	5
2	4	1	2
3	3	4	4
4	4	5	4

```
Out[4]: a    15
        b    15
        c    16
        dtype: int64
```

```
In [5]: df.apply(sum)
```

```
Out[5]: a    15
        b    15
        c    16
        dtype: int64
```

```
In [6]: df.sum()
```

```
Out[6]: a    15  
       b    15  
       c    16  
       dtype: int64
```

=> sum() 함수는 열 또는 행단위로 적용되는 함수(벡터화)

데이터프레임의 각 행에 sum() 함수 적용

```
In [7]: df  
df.apply(sum, axis=1)
```

```
Out[7]:
```

	a	b	c
0	1	2	1
1	3	3	5
2	4	1	2
3	3	4	4
4	4	5	4

```
Out[7]: 0      4  
       1     11  
       2      7  
       3     11  
       4     13  
       dtype: int64
```

```
In [8]: df.sum(axis=1)
```

```
Out[8]: 0      4  
       1     11  
       2      7  
       3     11  
       4     13  
       dtype: int64
```

데이터프레임의 각 원소의 제곱값을 계산

- 컬럼별 모든 원소에 대하여 np.square 함수 적용

```
In [9]: df  
df.apply(np.square)
```

```
Out[9]:
```

	a	b	c
0	1	2	1
1	3	3	5
2	4	1	2
3	3	4	4
4	4	5	4

```
Out[9]:
```

	a	b	c
0	1	4	1
1	9	9	25
2	16	1	4
3	9	16	16
4	16	25	16

- 행별 모든 원소에 대하여 np.square 함수 적용

```
In [10]: df.apply(np.square, axis=1)
```

```
Out[10]:
```

	a	b	c
0	1	4	1
1	9	9	25
2	16	1	4
3	9	16	16
4	16	25	16

사용자가 정의한 연산을 행/열단위 적용

apply() & lambda()

- 데이터프레임의 기본 집계함수(sum, min, max, mean 등)들은 행/열 단위 벡터화 연산을 수행함
 - apply() 함수를 사용할 필요가 없음
- apply() 함수 사용은 복잡한 연산을 해결하기 위한 lambda 함수나 사용자 정의 함수를 각 열 또는 행에 일괄 적용시키기 위해 사용
 - lambda 함수로 필요한 연산 기능을 구현하고, apply()를 통해 열/행 단위로 적용

1회성 함수 lambda 함수를 apply()에 사용

- 집합데이터의 최대값과 최소값의 차이를 구하는 lambda 함수 정의

```
In [11]: s = pd.Series([3,1,9,10,4])
s
```

```
Out[11]:
```

0	3
1	1
2	9
3	10
4	4

dtype: int64

```
In [12]: f1 = lambda x : x.max() - x.min()
```

- 정의한 lambda함수 f1() 적용

```
In [13]: f1(s)
```

Out[13]: np.int64(9)

- 데이터프레임에 적용

```
In [14]: # 각 컬럼의 최대값-최소값(범위:range) 계산
df
df.apply(f1, axis=0)
```

Out[14]:

	a	b	c
0	1	2	1
1	3	3	5
2	4	1	2
3	3	4	4
4	4	5	4

Out[14]:

a	3
b	4
c	4

dtype: int64

```
In [16]: df.apply(f1, axis=1)
```

Out[16]:

0	1
1	2
2	3
3	1
4	1

dtype: int64

- 직접 연산을 통해 각 행의 최대값과 최소값 차이 계산

```
In [17]: df.apply(lambda x:x.max()-x.min())
```

Out[17]:

a	3
b	4
c	4

dtype: int64

```
In [18]: df.max(axis=0) - df.min(axis=0)
```

Out[18]:

a	3
b	4
c	4

dtype: int64

apply 메서드에 전달된 함수가 시리즈를 반환하는 경우

- 스칼라값만 반환하지 않고, 여러 값을 가진 Series를 반환해도 됨

```
In [19]: def f2(x):
return pd.Series([x.min(), x.max()], index=['min','max'])
```

```
In [20]: df.apply(f2)
```

```
Out[20]:
```

	a	b	c
min	1	1	1
max	4	5	5

예. apply()를 이용한 데이터프레임 각 열의 데이터에 대한 범주별 빈도 계산

```
In [21]: df
```

```
Out[21]:
```

	a	b	c
0	1	2	1
1	3	3	5
2	4	1	2
3	3	4	4
4	4	5	4

```
In [22]: df.a.value_counts()
df.b.value_counts()
df.c.value_counts()
```

```
Out[22]: a
3      2
4      2
1      1
Name: count, dtype: int64
```

```
Out[22]: b
2      1
3      1
1      1
4      1
5      1
Name: count, dtype: int64
```

```
Out[22]: c
4      2
1      1
5      1
2      1
Name: count, dtype: int64
```

```
In [23]: df.apply(pd.Series.value_counts)
```

```
Out[23]:
```

	a	b	c
1	1.0	1	1.0
2	NaN	1	1.0
3	2.0	1	NaN
4	2.0	1	2.0
5	NaN	1	1.0

- 각 열의 데이터에 대한 범주별 빈도 계산 후, NaN값은 0으로 변환하고 전체 데이터 타입을 정수로 변환

```
In [24]: df.apply(pd.Series.value_counts).fillna(0).astype(int)
```

```
Out[24]:
```

	a	b	c
1	1	1	1
2	0	1	1
3	2	1	0
4	2	1	2
5	0	1	1

데이터프레임이나 시리즈 배열의 각 원소에 파이썬 함수를 적용하는 경우

- **map()** 사용 : 3.X 버전부터 사용
- pandas 2.X.X 버전의 경우 applymap()
 - Series는 각 원소에 적용할 함수를 지정하기 위해 map 메서드를 갖음

```
In [25]: df2 = pd.DataFrame(np.random.randn(4,3), columns=list('bde'), index='A B C D'.split())
df2
```

```
Out[25]:
```

	b	d	e
A	-1.792059	0.813309	-1.876614
B	2.016105	-0.212259	0.131790
C	0.692631	2.446788	-0.396181
D	-0.473670	-0.506331	-0.505918

```
In [26]: df2.map(lambda x: f'{x:.2f}')
```

```
Out[26]:
```

	b	d	e
A	-1.79	0.81	-1.88
B	2.02	-0.21	0.13
C	0.69	2.45	-0.40
D	-0.47	-0.51	-0.51

```
In [27]: df2.map(lambda x: f'{x:.2f}').info()
```

```
<class 'pandas.DataFrame'>
Index: 4 entries, A to D
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype 
---  -
0    b      4 non-null      str   
1    d      4 non-null      str   
2    e      4 non-null      str   
dtypes: str(3)
memory usage: 128.0+ bytes
```

```
In [29]: # map() : 시리즈 데이터의 각 원소에 대한 함수 적용
df2.b.map(lambda x: f'{x:.2f}')
```

```
Out[29]: A    -1.79
        B     2.02
        C     0.69
        D    -0.47
        Name: b, dtype: str
```

apply(), map()의 차이점

- apply() : 조건 기반 컬럼 생성, 여러 컬럼 조합 계산을 위해 사용
- map() : 단순 값 치환, 시리즈의 요소/데이터프레임의 전체 값 변환
- 벡터기반연산보다는 성능은 떨어짐 : 느린 방식

함수	대상	특징	비고
map	Series	1:1 매핑	pandas 2.X
apply	Series/DataFrame	함수 적용	
applymap	DataFrame	모든 요소에 적용	pandas 2.X까지 사용, 3.0이상에서는 deprecated -> map으로 변경

```
In [30]: df3 = pd.DataFrame({'grade': ['A', 'B', 'A', 'C']})
df3
```

```
Out[30]:
```

	grade
0	A
1	B
2	A
3	C

```
In [31]: # 등급을 숫자화
df3['grade_num'] = df3['grade'].map({'A':4, 'B':3, 'C':2})
df3
```

```
Out[31]:
```

	grade	grade_num
0	A	4
1	B	3
2	A	4
3	C	2

```
In [32]: df3['lower'] = df3['grade'].map(str.lower)
df3
```

```
Out[32]:
```

	grade	grade_num	lower
0	A	4	a
1	B	3	b
2	A	4	a
3	C	2	c

```
In [33]: df4 = pd.DataFrame({'math': [90, 85, 70],
                             'eng': [85, 90, 75]})
```


df4

```
Out[33]:
```

	math	eng
0	90	85
1	85	90
2	70	75

```
In [34]: # 조건 기반 컬럼 생성
df4['math'].apply(lambda x : 'A' if x>=90 else 'B')
```

```
Out[34]:
```

0	A
1	B
2	B

Name: math, dtype: str

```
In [35]: # 여러 컬럼 조합 계산
df4.apply(lambda x: (x['math'] + x['eng'])/2, axis=1)
```

```
Out[35]:
```

0	87.5
1	87.5
2	72.5

dtype: float64

```
In [36]: (df4['math'] + df4['eng'])/2
```

```
Out[36]:
```

0	87.5
1	87.5
2	72.5

dtype: float64

- Series.map() : 개별 원소에 함수 적용, 값 변환, 매핑
 - Series.apply() : 개별 원소에 함수 적용, 복잡한 함수 적용
 - DataFrame.map() : 개별 원소에 함수 적용, 데이터프레임의 모든 셀 변환
 - DataFrame.apply() : 행 또는 열 단위 함수 적용, 행/열 단위 연산
-